Angular Microfrontend Application

Application Flow, Architecture, and Demonstration Guide

Prepared: 22 October 2026 | **Validation:** TypeScript checks passed for all three projects using `tsc --project tsconfig.app.json --noEmit`. **Scope note:** The application was not exercised in a browser during this analysis.

1. What this application demonstrates

This is an Angular microfrontend application using `@angular-architects/native-federation`. Three separately built applications cooperate to manage one todo list and two counters.

These are frontend applications, rather than backend microservices. The repository contains no backend API, database, authentication, or persistent storage implementation.

Application	Development server	Responsibilities
Project A	http://localhost:4200	Host shell, navigation, shared state service, todo creation and completion, and counter summary
Project B	http://localhost:4201	Remote component, todo text editing, and counter B
Project C	http://localhost:4202	Remote component, todo deletion, and counter C

For the shared-state demo, open Project A and use its navigation. A displays B and C inside its router outlet.

2. Architecture

```
User → Project A shell and Angular Router
Project A → Home | Project B remote | Project C remote
Home, B, and C → AppStateService in Project A
AppStateService → RxJS BehaviorSubject
BehaviorSubject → state$ and AsyncPipe → active view
```

Only the active route's component is displayed. The state service survives route changes because it belongs to A's root application injector.

3. Federation configuration

Each project's `federation.config.js` configures its federation name, exposed modules, remote addresses, and shared dependencies.

Project	Exposed name	Source
A	`./AppStateService`	`src/app/app-state.service.ts`
B	`./Component`	`src/app/app.ts`
C	`./Component`	`src/app/app.ts`

A declares B and C as remotes. B and C declare A as a remote. The explicit runtime calls in the route and component files also supply the remote URL, name, and exposed module.

`remoteEntry.json` describes the remote's exposed modules and dependencies so the federation runtime can locate and load the requested code. It contains federation metadata, not todo data.

All projects use:

```
shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' })
```

- `singleton` requests a shared dependency version in the composed runtime.
- `strictVersion` enforces version compatibility during dependency sharing.
- `requiredVersion: 'auto'` derives the requirement from package metadata.
- `ignoreUnusedDeps: false` disables the optimization that ignores unused dependencies.
- `skip` excludes the listed dependency entry points from federation sharing.

Sharing dependency modules does not share JavaScript object instances between browser tabs. In the composed application, B and C share A's actual service object because the code retrieves it from `window.__appStateService`.

4. Startup flow

1. The browser loads A's `src/main.ts`.
2. `initFederation()` initializes the federation runtime before Angular bootstrapping.
3. `main.ts` dynamically imports `bootstrap.ts`.
4. `bootstrapApplication(App, appConfig)` starts A.
5. `app.config.ts` registers the routes using `provideRouter(routes)`.

6. A's root App injects `AppStateService` and assigns it to `window.__appStateService`.
7. Angular creates the root service on first injection. Its `BehaviorSubject` starts with `{ countB: 0, countC: 0, todos: [] }`.
8. The empty route loads A's `HomeComponent`.
9. Home renders the counters and the Todo component. Both read `state$` through `AsyncPipe`.

The startup error handler logs federation initialization errors and then continues to the bootstrap import. This does not guarantee that a failed remote can subsequently load.

5. Navigation and remote loading

Project B

1. The user clicks Project B in A's navigation.
2. Angular Router changes the URL to http://localhost:4200/project-b without a full page reload.
3. The route's `loadComponent` calls `loadRemoteModule` with B's http://localhost:4201/remoteEntry.json and `./Component`.
4. Federation loads the exposed component code; the route resolves to its exported App class.
5. Angular creates B's component inside A's `<router-outlet>`. It does not bootstrap B as a second complete application in this path.
6. B's `ngOnInit()` finds `window.__appStateService` and uses A's existing service object.
7. B reads the latest state through `state$ | async` and displays its counter and task editor.

Project C follows the same sequence using port 4202 and route `/project-c`.

The remote component is requested when the route is entered. Federation can also fetch metadata and shared dependencies during initialization, so avoid claiming that every federation network request is deferred until navigation.

6. State and business actions

```
interface AppState {
  countB: number;
  countC: number;
```

```
  todos: { id: number; text: string; completed: boolean }[];
}
```

`AppStateService` owns a private `BehaviorSubject`. `state$` exposes its observable side. A `BehaviorSubject` retains the current value and immediately supplies it to a new subscriber, which explains why returning to a route displays the latest data.

Every action follows this path:

```
UI event -> component method -> AppStateService method
         -> read current state -> construct updated state
         -> stateSubject.next(newState)
         -> active AsyncPipe subscriptions receive it -> UI updates
```

User action	Component method	Service method	State change
Add in A	`Todo.addTask()`	`addTodo(text)`	Trim text, reject blank input, append a task with `Date.now()` ID and `completed: false`
Complete/uncomplete in A	`Todo.toggleTask(id)`	`toggleTodo(id)`	Flip the matching task's `completed` value
Increment/decrement/reset in B	`increment/decrement/reset`	`updateCounter('b', action)`	Change `countB`
Start editing in B	`startEditing(task)`	None	Copy text into local `draftText`; shared state stays unchanged
Save in B	`saveTask(id)`	`editTodo(id, text)`	Replace matching

			task text, then clear local editing state
Cancel in B	`cancelEditing()`	None	Discard the local draft
Increment/decrement/reset in C	`increment/decrement/reset`	`updateCounter('c', action)`	Change `countC`
Delete in C	`deleteTask(id)`	`deleteTodo(id)`	Filter the task out of the array

Both counters allow negative numbers. Task changes use array spread, `map`, or `filter` to produce updated state. No HTTP request is involved in these actions.

7. Standalone mode versus the shared demo

Opening `localhost:4201` or `localhost:4202` directly bootstraps that application's own root.

In that case, `window.__appStateService` from A is absent. B/C fall back to loading A's `./AppStateService` through federation and resolving the class through their own Angular injector.

That creates a service instance in the standalone application's runtime. It does not connect to the instance in another tab. A, B, and C opened as separate pages therefore do not synchronize their counters or task lists.

Use the following addresses for the composed demo:

http://localhost:4200/
http://localhost:4200/project-b
http://localhost:4200/project-c

Navigate using the links inside A. A full reload, including a reload on a remote route, recreates the service and resets the data.

8. Demo setup and sequence

Start each application in its own terminal. Dependencies already exist in this workspace; for a fresh checkout, run `npm ci` in each project first.

```
# Terminal 1
cd C:\Users\Sakthi\Downloads\Project-angular\Project-A
npm start
# Terminal 2
cd C:\Users\Sakthi\Downloads\Project-angular\Project-B
npm start
# Terminal 3
cd C:\Users\Sakthi\Downloads\Project-angular\Project-C
npm start
```

Wait for all three servers to be ready. A's underlying dev-server configuration specifies port 4200; verify the actual printed address because its federation serve wrapper has `port: 0`, while the code's remote URLs expect A on 4200.

Step	Action	Expected result
1	Open Project A at port 4200	Empty task list; both counters show 0
2	Add `Prepare federation demo` and `Review shared state`	Two tasks appear
3	Complete `Prepare federation demo`	The first task is styled as completed
4	Select Project B from Project A's navigation	The same tasks and completion styling appear
5	Increment counter B twice	Counter B becomes 2
6	Rename `Review shared state` to `Explain RxJS shared state`, then save	The updated text appears
7	Return to Home	The edited text, completion status, and counter B value remain
8	Select Project C	Both current tasks appear
9	Increment counter C once and delete the completed task	Counter C becomes 1; one task remains
10	Return to Home, then Project B	Both views show the remaining edited task; Home shows B = 2 and C = 1

If useful, show the browser Network panel filtered by `remoteEntry` and the Console messages that say the service was loaded from `window`. Do not promise a fresh network fetch on every navigation because modules can be cached.

9. Suggested spoken explanation

“This application demonstrates Angular microfrontends using Native Federation. Project A is the host, and Projects B and C are separately served remote applications. A loads their exposed components through Angular Router.

“The business example is a shared todo workflow. A creates and completes tasks, B edits the text, and C deletes tasks. B and C also maintain separate counters, which A summarizes.

“The shared state lives in an RxJS BehaviorSubject inside A's AppStateService. A exposes its service instance on the browser window, and the remote components use that same instance when running inside A. Actions emit a new state, and AsyncPipe updates the active view. When I navigate between routes, the root service survives, so the latest values remain available.

“Federation handles loading independently built frontend code. RxJS handles state updates. This prototype keeps data in memory within the composed browser application.”

10. Questions and limitations to explain accurately

- **Is this a backend microservices example?** It demonstrates frontend microfrontends. No backend services are implemented here.
- **Why split into three projects?** Each has its own source, dependencies, build, and exposed module. Federation composes them at runtime. This repository uses fixed localhost addresses; production deployment automation is not included.
- **Does federation itself synchronize state?** No. It loads code and coordinates shared dependencies. The explicit shared service object and RxJS implement this application's state flow.
- **Why does data survive navigation?** A's root service instance survives while Angular replaces routed components.
- **Does data survive refresh?** No. The current active state is in memory, with no persistence.
- **Does it sync across tabs or users?** No. There is no active cross-tab transport or backend synchronization.

- **Is edit/delete ownership secured?** The UI distributes operations across projects, but there is no authorization enforcement. The shared service exposes all mutations.
- **What if a remote is down?** Navigation depends on that remote being available. There is no dedicated route-load error screen or retry flow. Standalone service loading catches errors and logs them.
- **What if versions conflict?** Strict sharing can reject incompatible dependency versions. Keep the applications' shared dependencies compatible.
- **Are there implementation shortcuts?** B/C use any for the service and $any in templates despite having local interface declarations. The window global also couples the remotes to the host's runtime convention. `Date.now()` IDs can collide for additions in the same millisecond.
- **Why do the READMEs mention SharedWorker?** They describe a different design. `public/state-worker.js` and the bridge assets still exist and are served, but the current Angular service never connects to them. The READMEs also refer to `./AppState`, while the actual exposed key is `./AppStateService`. Follow the active TypeScript code for this demo.

11. Code walkthrough order

1. `Project-A/federation.config.js`, then B/C configs: names, exposes, remotes, sharing.
2. `Project-A/src/main.ts` and `src/bootstrap.ts`: federation before Angular startup.
3. `Project-A/src/app/app.config.ts` and `app.routes.ts`: router and remote component loading.
4. `Project-A/src/app/app.ts`: inject and expose A's service instance.
5. `Project-A/src/app/app-state.service.ts`: state shape, BehaviorSubject, mutations.
6. `Project-A/src/app/home.component.ts` and `todo.ts`: display, creation, completion.
7. `Project-B/src/app/app.ts`: shared instance lookup, standalone fallback, edit/counter actions.
8. `Project-C/src/app/app.ts`: same lookup, delete/counter actions.
9. B/C `app-state.ts`: TypeScript contracts only; no independent store implementation.